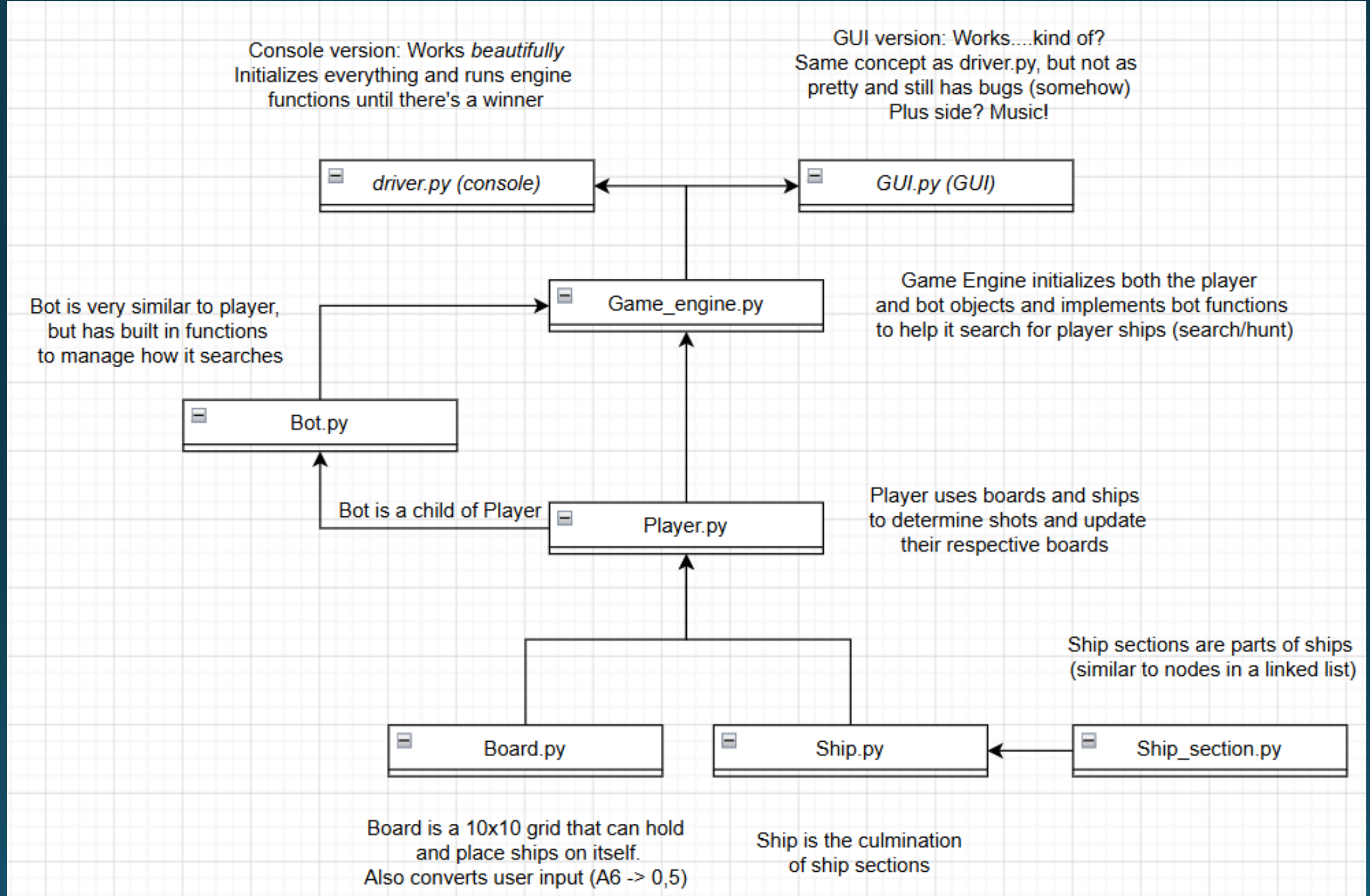

Battleship!
-Braxton Rider

In the beginning…

- For my project, I thought it would be fun and easy to code the hit game Battleship using Python.
- Players would be able to place and arrange their ships
- Player plays against a bot to sink their opponent's ships.
- Bot searches the board in two different states:
  - Search: It has no idea where the next ship is and continues down diagonal lines to try and find one.
  - Hunt: The bot hit a ship recently and is going to check all surrounding quadrants until a ship is sunk. Afterwards, it reverts back to searching.
- Game ends when all on an opponent's ships are sunk


Console version: Works *beautifully*
Initializes everything and runs engine functions until there's a winner
GUI version: Works....kind of?
Same concept as driver.py, but not as pretty and still has bugs (somehow)
Plus side? Music!
driver.py (console)
GUI.py (GUI)
Game_engine.py
Game Engine initializes both the player and bot objects and implements bot functions to help it search for player ships (search/hunt)
Bot is very similar to player, but has built in functions to manage how it searches
Bot.py
Bot is a child of Player
Player.py
Player uses boards and ships to determine shots and update their respective boards
Ship sections are parts of ships (similar to nodes in a linked list)
Board.py
Ship.py
Ship_section.py
Board is a 10x10 grid that can hold and place ships on itself.
Also converts user input (A6 -> 0,5)
Ship is the culmination of ship sections

Ship section

```
#This Defines a section of the ship
#This will be an object that is stored within a list of objects,
#Which combined together will be a ship
#Adding a next DA so I can implement this like a linked list
class ship_section:

    __is_hit = False
    __next = None

    #This will the same name as the ship that makes it
    __part_of = ""

    #Every ship part generated will be defaulted as not hit
    #This value will only change once the game starts running
    #So I'm just going to hard set it to False
    def __init__(self, ship_name):
        self.set_is_hit(False)
        self.set_next(None)
        self.set_part_of(ship_name)
```

```
#Helpers
#============================ BUILD SHIP OBJECT ============================================================
def build_ship(self):
    """Build Ship object using pre-defined name and length when Initializing object"""

    #List that will be set to the ship_sections
    ship_sections = []

    #For as many ship parts that are needed
    for x in range(self.get_length()):

        #Make new parts and add them to the ship list
        ship_sections.append(ship_section(self.get_name()))

    #Set the beginning of the ship as the bow and the end as the stern
    self.set_bow(ship_sections[0])

    self.set_stern(ship_sections[self.get_length() - 1])

    #After building, link sections together using the next DA in ship sections
    current_index = 0
    while (current_index < self.get_length() - 1):
        ship_sections[current_index].set_next(ship_sections[current_index + 1])
        current_index+=1

    #After building and setting ship sections, set ship
    self.set_ship_sections(ship_sections)

#============================ HIT SHIP SECTION ============================================================
def hit_ship_section(self, index):
    """Change the value of a ship section to be True (hit)"""
    modified_ship = self.get_ship_sections()
    modified_ship[index].set_is_hit(True)

    #After ship is hit, do a check to see if it has been sunk
```

Ship

Board

```
#============================ SET SHIP ON GRID ============================================
def set_ship_on_grid(self, ship = None, quadrant = "", axis = 0):

    """Take in a ship object and try to place it on the grid using the starting index and axis\n
    For the starting index, it will be a string that will be parsed (A6)\n
    For the axis value, 0 is horizontal and 1 is vertical"""

    #Firstly, find the exact coordinates of the ship using the starting index
    #Assume that the starting index is within the bounds of the board (player object will handle verifying that)

    #Coordinate values that will be determined
    coordinates = self.convert_input(quadrant)

    #Flag to help determine success of ship placement
    error_value = 0

    #If no errors so far
    if (coordinates[0] != -1):
        #If the ship is horizontal
        if (axis == 0):

            #Flag value to see if the ship positioning is good
            valid = True

            #Check if there's enough room for the ship horizontally
            #Because the ship is occupying the first space, we can go up to ten
            #Another way to say this is if we subtracted 1 from ship length and it was greater than 9
            if (coordinates[1] + ship.get_length() > 10):
                print("ERROR: Ship is out of bounds of Board Horizontally")
                error_value = 1
                valid = False
```

```
def determine_shot(self, quadrant):

    """A shot will be taken on this entity's board to determine if it hits a ship"""

    #Flag value that determines shot
    result = -1

    #Get User Input will help verify this before it runs into this function
    coordinates = self.get_my_ship_board().convert_input(quadrant)
    updated_grid = self.get_my_ship_board().get_grid()
    current_tile = self.get_my_ship_board().get_grid()[coordinates[0]][coordinates[1]]


    #IF MISS
    if (current_tile == None or current_tile == "~0~~"):
        print("\n\t\t ~~~ M ~~~ M ~~~ M~~~ MISS ~~~ M ~~~ M ~~~ M ~~~")
        updated_grid[coordinates[0]][coordinates[1]] = "~0~~"

        #If miss
        result = 0

    #IF RE-HIT
    elif (current_tile == "~X~~"):
        print("\n\t\t    ??? ?? DAMAGED PART WAS HIT.. AGAIN ?? ???")

        #If damaged part was hit again for some reason
        result = 1

    #IF HIT
    else:
        print("\n\t\t! ! ! ! ! ! ! ! ! | HIT ! | ! ! ! ! ! ! ! ! ! !")

        #Find the name of the ship that was hit
        current_ship_index = 0
```

Player

```python
#Helpers
#================ GENERATE SEARCH SPACE ================================================================================
def generate_search_space_static(self):
    """Generate a seach space for the bot."""
    temp_space = []

    count = 0
    column_counter = 0

    #For the first half of quadrants (5 x 10)
    for x in range(5):
        for y in range(10):
            #Add them to the search space
            temp_space.append(f"{str(self.get_my_ship_board().get_column_markers()[(x + y + column_counter) % 10])}" + f"{str(self.get_my_ship_board().get_row_markers()

        count += 1
        column_counter += 1

    count = 0
    column_counter = 1

    #For the second half of the quadrants (5 x 10)
    for x in range(5):
        for y in range(10):
            #Add them to the search space
            temp_space.append(f"{str(self.get_my_ship_board().get_column_markers()[(x + y + column_counter) % 10])}" + f"{str(self.get_my_ship_board().get_row_markers()

        count += 1
        column_counter += 1

    #print(temp_space)
    self.set_search_space(temp_space)
```

Bot

```
def ship_hunt_static(self):
    """Hunt down a ship once one is detected, return a quadrant to shoot. Will hunt clockwise [L-T-R-B]"""

    #Search the surrounding quadrants
    hunt_queue = self.get_bot().get_hunt_queue()

    #In case the hunt queue is somehow empty
    #Most likely caused by ship stacking
    #This usually doesn't happen, but just in case
    if (len(hunt_queue) == 0):
        guess = self.get_bot().get_search_space()[0]

    else :
        guess = hunt_queue.pop(0)

    self.get_bot().set_hunt_queue(hunt_queue)

    #Also add these guesses to the previously guessed list
    temp_list = self.get_bot().get_previous_guesses()
    temp_list.append(guess)
    self.get_bot().set_previous_guesses(temp_list)

    #Finally, remove this guess from the static search list
    search_space = self.get_bot().get_search_space()
    search_space.remove(guess)
    self.get_bot().set_search_space(search_space)


    return guess
```

Game engine

Driver

```python
#Flag to determine who wins the game
who_won = -1
game = game_engine()
game.set_player_board()
game.set_bot_board()
#print(game.get_bot().get_my_guess_board())

game.get_bot().generate_search_space_static()

count = 0

is_running = True
#While the game is running
while is_running:

    count += 1
    #================ PLAYER ================================
    ui = game.get_player().get_user_input("Enter a Quadrant [A5, B10, etc]: ")
    shot_determination = game.get_bot().determine_shot(ui)
    game.get_player().update_guess_board(ui, shot_determination)


    #Get string
    output_string = game.end_game(game.determine_winner())

    #Determine if player won
    if output_string != "":

        is_running = False
        print(output_string)
        continue
```

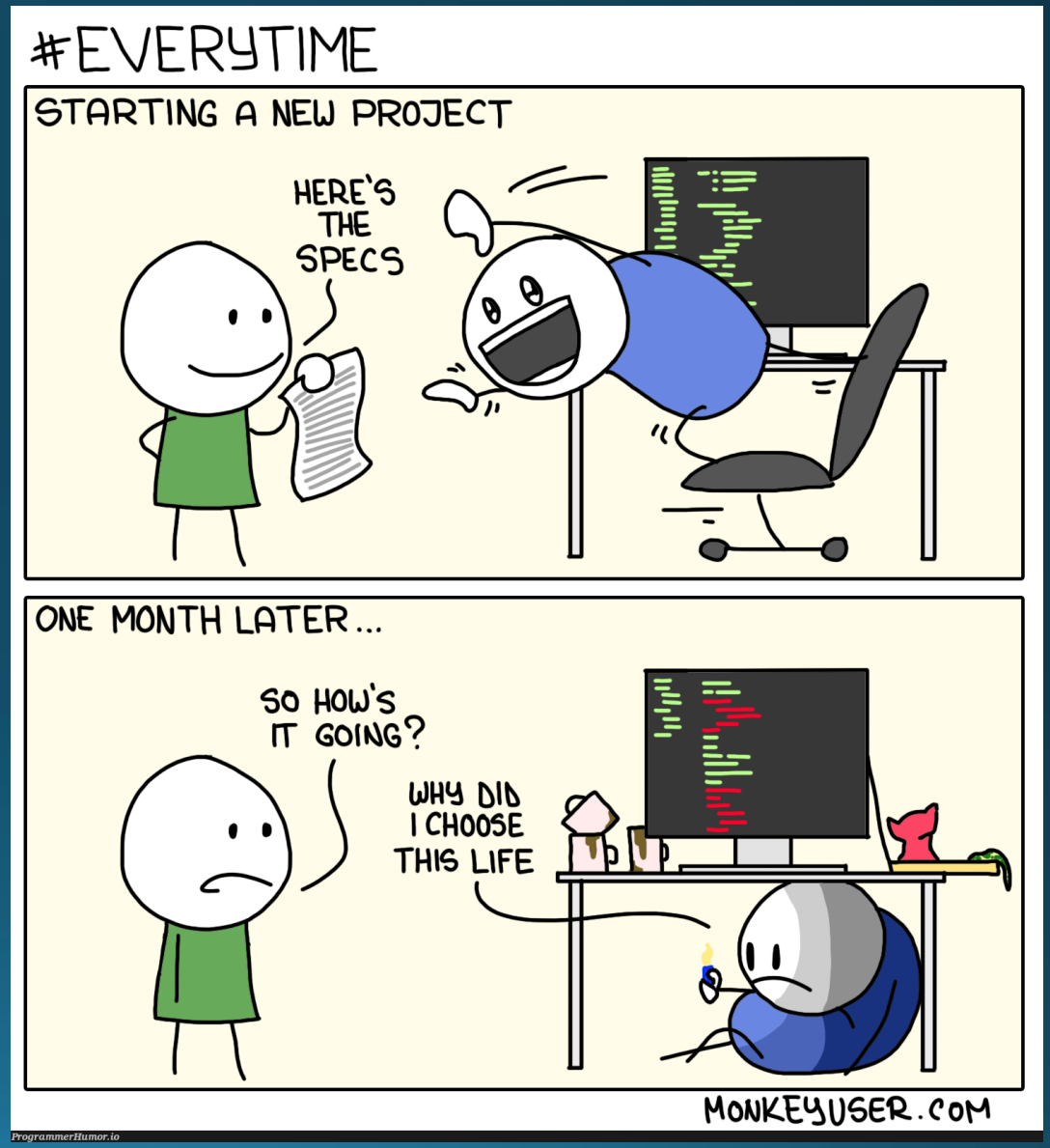
```python
#================= BOT ================================

#Determine current state of bot

#If hunting
if (game.get_bot().get_is_hunting()):
    bot_guess = game.ship_hunt_static()
    shot_determination = game.get_player().determine_shot(bot_guess)

    #This will determine whether the bot keeps hunting or switches back to searching
    game.get_bot().update_hunt_queue(shot_determination)

#If searching
elif (not game.get_bot().get_is_hunting()):
    bot_guess = game.ship_search_static()
    shot_determination = game.get_player().determine_shot(bot_guess)

    #If the bot hits a ship, switch to hunting mode
    if shot_determination == 3:
        game.get_bot().set_is_hunting(True)
        game.get_bot().update_hunt_queue(shot_determination)


game.get_bot().update_guess_board(bot_guess, shot_determination)

#Get string
output_string = game.end_game(game.determine_winner())

#Determine if bot won
if output_string != "":

    is_running = False
    print(output_string)
    print(count)
    continue
```

```
self.__main_frame = ttk.Frame(self.__root)
self.__main_frame.pack(fill=tk.BOTH, expand=True)

self.__main_frame.columnconfigure(0, weight=10)
self.__main_frame.columnconfigure(1, weight=0)
self.__main_frame.columnconfigure(2, weight=10)
self.__main_frame.rowconfigure(0, weight=5)

#Left window
left_frame = ttk.LabelFrame(self.__main_frame, text="Player Board")
left_frame.grid(row=0, column=0, sticky="nsew")
self.__left_window = ScrolledText(left_frame, wrap=tk.WORD, font=("Arial", 12))
self.__left_window.pack(fill=tk.BOTH, expand=True)

#Center window
center_frame = ttk.LabelFrame(self.__main_frame, text="Game Messages")
center_frame.grid(row=0, column=1, sticky="nsew")
self.__center_window = ttk.Label(center_frame, anchor="center", justify="center", font=("Arial", 12, "bold"), wraple
self.__center_window.pack(expand=True, fill=tk.BOTH, padx=10, pady=10)

#Right window
right_frame = ttk.LabelFrame(self.__main_frame, text="Guess Board")
right_frame.grid(row=0, column=2, sticky="nsew", padx=5)
self.__right_window = ScrolledText(right_frame, wrap=tk.WORD, font=("Arial", 12))
self.__right_window.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

#Input
input_frame = ttk.Frame(self.__root, padding=(10, 20, 10, 10))
input_frame.pack(fill=tk.X)
self.__input_var = tk.StringVar()
self.__user_input = ttk.Entry(input_frame, textvariable=self.__input_var, font=("Arial", 12))
self.__user_input.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 10))
self.__submit_button = ttk.Button(input_frame, text="Submit", command=self.submit_input)
self.__submit_button.pack(side=tk.RIGHT)
self.__user_input.bind("<Return>", lambda event: self.submit_input())
```

GUI (gross)

Challenges faced:

- Procrastination.
- Confusing myself with row/column names (because it logically should be row A, column 6 and not row 6, column A).
- GUI…just…all of it really.



Quick little demo time